NurseIO - Wingspan API Integration Guide

For: Josh Keeley (CTO), Lemuel, Arun | **Prepared by:** Jason Krigsfeld, Wingspan SE

Staging API: `https://stagingapi.wingspan.app`

Prod API: `https://api.wingspan.app`

Staging UI: `https://staging-my.wingspan.app`

Prod UI: `https://my.wingspan.app`

Staging Environment

NurseIO's staging tenant is live. Below is how your account structure will look once configured.

Account Structure (Two-Bucket Pattern)

Your org uses two types of child accounts under the root parent — one bucket for your contractors and separate child accounts for each of your entities. This is the recommended pattern for multi-entity payers whose contractors work across entities.

```
Slink Health (Root Parent)
├── Payee Bucket (Child User)         ← All contractors live here. Onboard once.
│   ├── Nurse Jane Smith
│   ├── Nurse John Doe
│   └── ... (all contractors)
│
├── NurseIO AZ (Entity Child User)    ← EIN #1 — payables created here
├── NurseIO KS (Entity Child User)    ← EIN #2 — payables created here
├── NurseIO TX (Entity Child User)    ← EIN #3 — payables created here
├── Entity 4 (Entity Child User)      ← EIN #4
└── Entity 5 (Entity Child User)      ← EIN #5
```

Account	Role	Description
Slink Health (Root Parent)	Parent Org	Root organization — API tokens generated here. Config cascades to all children.
Payee Bucket (Child User)	Contractor Pool	All contractors created and onboarded here. Onboard once (W-9, payout method). Org config auto-assigns them as child users.
NurseIO AZ, KS, TX... (Entity Child Users)	Entity / EIN Buckets	One per legal entity / EIN. Payables created here so the correct EIN is tied to the payment and 1099 filing.

Why Two Buckets?

- **Contractors onboard once.** POST /payments/payee from the Payee Bucket context — contractor creates account, fills out W-9, sets up payout method. Done. Never again.

- **Payables tie to the correct EIN.** When a nurse clocks out under NurseIO AZ, you create the payable from the AZ entity context. That payment is associated with AZ's EIN for 1099 purposes.
- **Cross-entity relationships are lightweight.** When a contractor needs to work under a new entity, call POST /payments/payee again from that entity's context with the same email. Creates the relationship without forcing re-onboarding.

Key Concepts

- **Scoped per child.** Payables created under NurseIO AZ are tied to AZ's EIN. Payables under NurseIO KS → KS's EIN.
- **Configuration inheritance flows downward.** Branding, terminology, support emails set on root parent are inherited by all children. Configure once.
- **Org config drives automatic child user assignment.** Payees created under the Payee Bucket automatically become child users. No manual step.

Staging Credentials

```
Email:    nursio+staging@m.wingspan.app
Password: Wingspan123!
```

Log in at `https://staging-my.wingspan.app` with the credentials above. From there you can switch between org contexts and generate your API token.

Staging Environment Notes

- 2FA can be bypassed in staging — any 6-digit code works
- Use random/fake numbers for EIN/SSN in staging
- No real emails or PDFs are generated in staging
- Staging data does not affect production

Getting Your API Token

1. Log into staging-my.wingspan.app
2. Switch to the Slink Health org (root parent)
3. Navigate to Data & Integrations > Developer
4. Click Generate new token — select "No expiration" for staging
5. Copy the token immediately — treat it like a password

Your requests will include these headers:

```
Authorization: Bearer {YOUR_API_TOKEN}
Content-Type: application/json
```

Two Ways to Authenticate: Parent Token vs. Child Token

Every API call in this guide shows both options. Most clients use Option A since it requires one token and lets you switch between children (Payee Bucket, NurseIO AZ, NurseIO KS, etc.) with a header swap.

Option A: Parent Token + X-WINGSPAN-USER Header (Recommended)

Use a single API token from the root parent (Slink Health). Specify which child you're acting as via the X-WINGSPAN-USER header.

```
Authorization: Bearer {SLINK_HEALTH_PARENT_API_TOKEN}
X-WINGSPAN-USER: {CHILD_USER_ID}
Content-Type: application/json
```

This is especially powerful for your setup — one token, and your backend just changes the X-WINGSPAN-USER header depending on whether you're creating a payee (Payee Bucket) or creating a payable (entity like NurseIO AZ).

Option B: Direct Child API Token (No Header Needed)

Generate a token directly from each child account. No header needed — but you'd need a separate token per child.

```
Authorization: Bearer {CHILD_API_TOKEN}
Content-Type: application/json
```

When to Use Which

Scenario	Recommended Auth
Single backend managing multiple entities	Option A — one token, switch child via header
Scoped access for a specific team member	Option B — token is inherently scoped
Automated systems / backend services	Option A — easier key rotation
Per-entity microservices (unlikely for you)	Option B — one token per service

Important: Resources created in one child are NOT visible in another. A payee created under the Payee Bucket is not visible from NurseIO AZ until you establish a payer-payee relationship there too.

Part 1: Organization Configuration & Inheritance

This section documents how the two-bucket org structure is built and how configuration cascades. Wingspan sets this up during onboarding, but it's helpful to understand the mechanics — especially if you need to add entities later.

Creating Child Users (Prerequisite)

Before association, child users must be created via the organization user endpoint. This sets proper ownership so they can be associated with the parent.

```
POST https://stagingapi.wingspan.app/users/organization/user
```

```
curl -X POST https://stagingapi.wingspan.app/users/organization/user \
  -H "Authorization: Bearer {SLINK_HEALTH_PARENT_API_TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "nursio+az@m.wingspan.app",
    "profile": { "preferredName": "NurseIO AZ" }
  }'
```

The authenticated parent's token automatically sets ownership on the new user — this is what makes the subsequent /associate call work. Repeat for each child: Payee Bucket, NurseIO AZ, NurseIO KS, etc.

Important: Use `POST /users/organization/user` (not `POST /users/user`). The organization endpoint sets `ownerIds` on the child so the parent can manage it. The base `/users/user` endpoint does not set ownership.

Associating Child Users with the Parent

Once child users are created, associate each with the root parent via:

```
POST
https://stagingapi.wingspan.app/users/organization/user/{childUserId}/associate
```

Where `{childUserId}` is the User ID of the child account being attached (e.g., the Payee Bucket or an entity like NurseIO AZ).

```
curl -X POST
https://stagingapi.wingspan.app/users/organization/user/{PAYEE_BUCKET_USER_ID}/asso
ciate \
  -H "Authorization: Bearer {SLINK_HEALTH_PARENT_API_TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{
    "parentUserId": "{SLINK_HEALTH_USER_ID}",
    "inheritanceStrategy": {
      "organizationAccountConfig": "Parent",
      "wingspanAccount": "None",
      "externalFinancialAccounts": "None",
```

```
      "wingspanFinancialSettings": "None",
      "fundingSource": "None",
      "payoutSettings": "None"
    }
  }'
```

Field	Type	Description
parentUserId	string	Required. The User ID of the parent account (Slink Health root user).
inheritanceStrategy	object	Controls which config cascades from parent to child. Each key is a strategy type: "Parent" (inherit) or "None" (independent).

Response

```
{
  "organizationId": "org_slink_health...",
  "parentUserId": "user_slink_health...",
  "childUserId": "user_payee_bucket...",
  "inheritanceStrategy": {
    "organizationAccountConfig": "Parent",
    "wingspanAccount": "None",
    "externalFinancialAccounts": "None",
    "wingspanFinancialSettings": "None",
    "fundingSource": "None",
    "payoutSettings": "None"
  }
}
```

Call this once per child — once for the Payee Bucket, once per entity. The result is the tree structure shown at the top of this guide.

Inheritance Strategy Options

Each strategy type controls what flows down from parent to child. Set to "Parent" to inherit, "None" to keep independent.

Strategy	What It Controls	NurseIO Setting
`organizationAccountConfig`	Branding, terminology, support emails, appearance	"Parent" — inherit from Slink Health
`wingspanAccount`	Core Wingspan account/member settings	"None" — each child manages its own
`externalFinancialAccounts`	Connected external bank accounts (Plaid)	"None" — each entity has its own bank
`wingspanFinancialSettings`	Financial configuration (fees, thresholds)	"None" — entity-specific
`fundingSource`	Which bank account to debit for payroll	"None" — each entity debits its own
`payoutSettings`	Payout method configuration	"None" — entity-specific

For NurseIO, the key inheritance is `organizationAccountConfig: "Parent"` — this makes branding, terminology, and support cascade from Slink Health to every entity and the Payee Bucket.

Everything else stays `"None"` because each entity has its own EIN, bank account, and funding source.

Setting Configuration on the Parent

Before associating children, set the branding, terminology, and support config on the **root parent** (Slink Health) — see Part 2. Any child associated with `organizationAccountConfig: "Parent"` automatically inherits these settings.

The config flows through the parent chain. Wingspan walks up the tree: child → parent → grandparent → root. At each level where organizationAccountConfig is "Parent", the parent's config is merged in.

Updating Inheritance After Association

If you need to change inheritance settings later:

```
PATCH
https://stagingapi.wingspan.app/users/organization/user/{childUserId}/associate

curl -X PATCH .../users/organization/user/{NURSEIO_AZ_USER_ID}/associate \
  -H "Authorization: Bearer {SLINK_HEALTH_PARENT_API_TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{ "inheritanceStrategy": { "fundingSource": "Parent" } }'
```

Adding a New Entity Later

If NurseIO expands to a new state (e.g., NurseIO FL):

6. `POST /users/organization/user` to create the NurseIO FL child user (sets ownership automatically)
7. `POST /users/organization/user/{FL_USER_ID}/associate` with parentUserId = Slink Health and organizationAccountConfig: "Parent"
8. Set the entity's EIN and bank account
9. Configuration (branding, terminology, support) inherits automatically — no reconfiguration
10. Start creating payee associations and payables under the new entity

How Inheritance Works in Practice

```
Slink Health (Root Parent)
│  Sets: branding (NurseIO logo), terminology, support emails, appearance
│
├─ Payee Bucket  [organizationAccountConfig: "Parent"]
│  Inherits: branding, terminology, support from Slink Health
│  Contractors see NurseIO branding in their onboarding flow
│
├─ NurseIO AZ  [organizationAccountConfig: "Parent"]
│  Inherits: branding, terminology, support from Slink Health
│  Own settings: AZ EIN, AZ bank account, AZ funding source
│
└─ NurseIO KS  [organizationAccountConfig: "Parent"]
   Inherits: branding, terminology, support from Slink Health
   Own settings: KS EIN, KS bank account, KS funding source
```

Key insight: Configure branding, terminology, and support **once** on the root parent. Every child with `organizationAccountConfig: "Parent"` inherits. Entity-specific settings (EIN, bank, funding) stay at "None" and are configured per entity.

What Inherits vs. What Doesn't

Inherits (organizationAccountConfig)	Per Entity (inheritance = "None")
Branding (logo, name, URL)	EIN / Tax ID
Terminology	Bank account (funding source)
Support emails	Financial settings (fees, thresholds)
Appearance / visual theming	Payout settings
Email customization	External bank accounts

Part 2: Appearance Config, Branding & Customization

Set on the root parent (Slink Health) — all children inherit automatically.

Endpoint

```
PATCH https://stagingapi.wingspan.app/users/customization/{SLINK_HEALTH_USER_ID}
```

Branding

Logo URLs, company name, and website:

```
{
  "branding": {
    "name": "NurseIO",
    "url": "https://nurseio.com",
    "primaryLogoUrl": "https://nurseio.com/logo.png",
    "secondaryLogoUrl": "https://nurseio.com/icon.png"
  }
}
```

Support

Important: the API requires both documentation and portal objects:

```
{
  "support": {
    "generalSupportEmail": "support@nurseio.com",
    "payeeSupportEmail": "payments@nurseio.com",
    "documentation": {
      "generalUrl": "https://help.nurseio.com"
    },
    "portal": {
      "generalUrl": "https://support.nurseio.com"
    }
  }
}
```

Terminology

Healthcare/staffing customization:

```
{
  "terminology": {
    "sendPaymentsContractor": "Healthcare Professional",
    "sendPaymentsPayable": "Shift Payment",
    "getPaidClient": "NurseIO"
  }
}
```

Organization Settings

This is the setting that makes the two-bucket pattern automatic. Set `defaultNewPayeeParentAccountId` to your Payee Bucket's User ID — Wingspan will then automatically make any new payee a child user of that bucket.

```
{
  "organizationSettings": {
    "defaultNewPayeeParentAccountId": "{PAYEE_BUCKET_USER_ID}"
  }
}
```

Field	Description
`defaultNewPayeeParentAccountId`	When a payee is created under this account (or its children), automatically assign them as a child of the specified parent account (the Payee Bucket).
`defaultNewPayerParentAccountId`	Same concept for payer accounts. Not needed for NurseIO's use case.

Important: Set this on the root parent (Slink Health) so it cascades. Without this setting, new payees created via `POST /payments/payee` won't automatically become child users of the Payee Bucket — the bucket pattern won't work.

Appearance

The appearance object controls visual styling across the contractor-facing UI. Set on the parent and it cascades to all children.

Top-level appearance properties:

Property	Type	Description
`colorPrimary`	string (hex)	Primary brand color (e.g., "#2B579A")
`borderRadius`	number	Border radius in pixels for buttons, inputs, cards
`fontFamily`	string	Font family (e.g., "Arial", "Helvetica")
`colorText`	string (hex)	Primary text color
`colorBorder`	string (hex)	Border color for inputs and UI elements

The appearance object also supports nested sub-objects (button, link, input, control, text, select) with fine-grained style overrides. See the full Appearance Config API Guide for comprehensive reference.

Example: Minimal Appearance Config

Just set the primary brand color:

```
{
  "appearance": {
    "colorPrimary": "#E74C3C"
  }
}
```

Example: Brand Color + Rounded Corners + Font

A more comprehensive appearance customization:

```
{
  "appearance": {
```

```
    "colorPrimary": "#2B579A",
    "borderRadius": 8,
    "fontFamily": "Segoe UI",
    "colorText": "#1F2937",
    "colorBorder": "#D1D5DB"
  }
}
```

Part 3: Custom Fields

Custom fields let NurseIO define structured metadata on payees (Collaborator), payable line items (LineItem), and engagements (Engagement). This metadata is stored and available for reporting, filtering, and reconciliation.

Endpoint

```
POST https://stagingapi.wingspan.app/payments/custom-fields
```

Creating a Custom Field

```
curl -X POST https://stagingapi.wingspan.app/payments/custom-fields \
  -H "Authorization: Bearer {SLINK_HEALTH_PARENT_API_TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Shift ID",
    "key": "shiftId",
    "type": "String",
    "required": false,
    "resourceType": "LineItem"
  }'
```

Request Fields

Field	Type	Required	Description
name	string	Yes	Human-readable display name for the field
key	string	Yes	Machine-readable key (used in labels, e.g., shiftId)
type	string	Yes	Data type: "String", "Number", "Boolean", "Datetime", or "ValueSet"
required	boolean	Yes	Whether this field is required when creating the resource
resourceType	string	Yes	Where field applies: "Collaborator" (payees), "LineItem" (line items), or "Engagement"

Response Example

```
{
  "id": "FouRJrgAIjiLD6nqQQgNlF",
  "name": "Shift ID",
  "key": "shiftId",
  "type": "String",
  "required": false,
  "resourceType": "LineItem"
}
```

Listing Custom Fields

```
GET https://stagingapi.wingspan.app/payments/custom-fields
```

Returns all custom fields defined for your organization.

Using Custom Fields

Once defined, pass custom field values via the labels object:

- **On payee creation:** `payerOwnedData.labels` in POST /payments/payee
- **On payable line items:** `lineItems[].labels` in POST /payments/payable

Recommended Custom Fields for NurseIO

Based on your shift-pay workflow:

LineItem fields:

- shiftId (String) — unique identifier for the shift
- facilityId (String) — facility or location ID
- payCode (String) — pay code (STANDARD, PREMIUM, OVERTIME, etc.)

Collaborator fields:

- discipline (String) — healthcare discipline (RN, LPN, CNA, etc.)
- homeState (String) — home state for regulatory compliance

Part 4: Creating a Payee (Initial Onboarding)

When a contractor activates in your system, call this endpoint from the Payee Bucket context. The contractor onboards once — W-9, payout method, done. This is the "seed the onboarding form" step Josh described on the kickoff call.

Endpoint

```
POST https://stagingapi.wingspan.app/payments/payee
```

Option A: Parent Token + X-WINGSPAN-USER (Payee Bucket)

```
curl -X POST https://stagingapi.wingspan.app/payments/payee \
  -H "Authorization: Bearer {SLINK_HEALTH_PARENT_API_TOKEN}" \
  -H "X-WINGSPAN-USER: {PAYEE_BUCKET_USER_ID}" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "nurse.jane@example.com",
    "firstName": "Jane",
    "lastName": "Smith",
    "payerOwnedData": {
      "payeeExternalId": "NIO-123456",
      "status": "Active",
      "labels": {
        "discipline": "RN",
        "homeState": "AZ"
      }
    }
  }'
```

Option B: Direct Child Token (Payee Bucket)

```
curl -X POST https://stagingapi.wingspan.app/payments/payee \
  -H "Authorization: Bearer {PAYEE_BUCKET_API_TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "nurse.jane@example.com",
    "firstName": "Jane",
    "lastName": "Smith",
    "payerOwnedData": {
      "payeeExternalId": "NIO-123456",
      "status": "Active"
    }
  }'
```

Request Body Fields

Field	Type	Req	Description
email	string	Yes	Contractor's email. If user exists, they're linked. Otherwise created and invited.
firstName	string	No	Contractor's first name
lastName	string	No	Contractor's last name

`payerOwnedData.payeeExternalId`	string	Rec.	Your internal contractor/resource ID. Use for idempotent lookups.
`payerOwnedData.status`	string	No	"Active" or "Inactive". Defaults to Active.
`payerOwnedData.labels`	object	No	Key-value metadata (discipline, state, facility, etc.)
`payerOwnedData.payeeW9Data`	object	No	Pre-populate W-9 info to seed the onboarding form.

Pre-Populating W-9 / Address Data (Seeding the Form)

Since you're collecting personal info during your 5-step wizard, pass it through to pre-populate Wingspan's onboarding form. The contractor sees pre-filled info and just confirms.

```
{
  "email": "nurse.jane@example.com",
  "firstName": "Jane",
  "lastName": "Smith",
  "payerOwnedData": {
    "payeeExternalId": "NIO-123456",
    "status": "Active",
    "payeeW9Data": {
      "country": "US",
      "addressLine1": "123 Desert Rd",
      "city": "Phoenix",
      "state": "AZ",
      "postalCode": "85001"
    }
  }
}
```

Response

Save the `user.userId` — you'll need it for session tokens. The `payeeExternalId` you set is echoed back in `payerOwnedData`.

```
{
  "payeeId": "payee_abc123...",
  "payerId": "user_payee_bucket...",
  "user": {
    "userId": "user_jane_smith_xyz...",
    "email": "nurse.jane@example.com"
  },
  "payerOwnedData": {
    "payeeExternalId": "NIO-123456",
    "status": "Active"
  }
}
```

What Happens Behind the Scenes

11. Wingspan creates the user (or links to an existing one)
12. A payer-payee relationship is established between the Payee Bucket and the new user
13. Org config makes the new user automatically a child user of the Payee Bucket
14. Contractor inherits all configuration (branding, terminology, support) from the parent chain

15. Contractor receives an invite email to complete onboarding (W-9, payout method)

Part 5: Associating a Payee with an Entity

Before you can create payables for a contractor under a specific entity (e.g., NurseIO AZ), you need to establish a payer-payee relationship between that entity and the contractor. Same endpoint as Part 4 — different child context.

The contractor does NOT re-onboard. They already have a Wingspan account from Part 4. This just creates the relationship so you can pay them from that entity.

Endpoint

```
POST https://stagingapi.wingspan.app/payments/payee
```

Option A: Parent Token + X-WINGSPAN-USER (Entity Context)

```
curl -X POST https://stagingapi.wingspan.app/payments/payee \
  -H "Authorization: Bearer {SLINK_HEALTH_PARENT_API_TOKEN}" \
  -H "X-WINGSPAN-USER: {NURSEIO_AZ_USER_ID}" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "nurse.jane@example.com",
    "payerOwnedData": {
      "payeeExternalId": "NIO-123456",
      "status": "Active"
    }
  }'
```

Notice the `X-WINGSPAN-USER` header now points to the **entity** (NurseIO AZ), not the Payee Bucket. Same token, different header — that's the power of Option A.

Option B: Direct Entity Token

```
curl -X POST https://stagingapi.wingspan.app/payments/payee \
  -H "Authorization: Bearer {NURSEIO_AZ_API_TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "nurse.jane@example.com",
    "payerOwnedData": {
      "payeeExternalId": "NIO-123456",
      "status": "Active"
    }
  }'
```

Response

Returns a new payer-payee relationship. The response includes a `requirements` array — **save the payerPayeeEngagementIds[0]** from any requirement. This is the default engagement ID and the `collaboratorId` you'll use in Part 6.

```
{
  "payeeId": "payee_az_def456...",
  "payerId": "user_nurseio_az...",
  "user": { "email": "nurse.jane@example.com", "status": "Pending" },
```

```
  "payerOwnedData": { "payeeExternalId": "NIO-123456", "status": "Active" },
  "requirements": [
    {
      "payerPayeeEngagementIds": ["ppe_az_jane_abc123..."],
      "requirementType": "Registration",
      "status": "Incomplete"
    }
  ]
}
```

Critical: The `payerPayeeEngagementId` (in `requirements[].payerPayeeEngagementIds[0]`) is the engagement-level relationship ID. NOT the same as `payeeId`. A default engagement is auto-created when the relationship is established. Save one per entity per contractor.

Example mapping to save:

Contractor	Entity	payerPayeeEngagementId (= collaboratorId)
Jane Smith	NurseIO AZ	ppe_az_jane_abc123...
Jane Smith	NurseIO KS	ppe_ks_jane_def456...
John Doe	NurseIO AZ	ppe_az_john_ghi789...

Note: `user.userId` is the same as Part 4 — same contractor. But `payerPayeeEngagementId` and `payeeId` are different because it's a different payer-payee relationship (NurseIO AZ ↔ Jane vs. Payee Bucket ↔ Jane).

When to Call This

- **On first shift assignment:** When a contractor is first assigned to work at a facility under NurseIO AZ, call this then. Lazy enrollment.
- **At onboarding time:** If you know which entities a contractor will work under upfront, call for each entity right after Part 4.
- **Bulk migration:** When migrating existing contractors, create entity relationships per their active assignments.

The 409 (duplicate) response is safe to ignore — the relationship already exists.

Part 6: Creating a Payable & Triggering Payment

The core of your integration. It's a two-step process: first you create individual payables as nurses clock out, then you trigger a bulk payroll batch to pay all approved payables at once. Payables are created from the entity context (not the Payee Bucket), so each is tied to the correct EIN.

Step 1: Create an Individual Payable

When a nurse clocks out via NFC, create a payable under the correct entity. The `collaboratorId` is the `payerPayeeEngagementId` you saved from Part 5.

Endpoint:

```
POST https://stagingapi.wingspan.app/payments/payable
```

Option A: Parent Token + X-WINGSPAN-USER (Entity Context)

```
curl -X POST https://stagingapi.wingspan.app/payments/payable \
  -H "Authorization: Bearer {SLINK_HEALTH_PARENT_API_TOKEN}" \
  -H "X-WINGSPAN-USER: {NURSEIO_AZ_USER_ID}" \
  -H "Content-Type: application/json" \
  -d '{
    "collaboratorId": "ppe_az_jane_abc123...",
    "dueDate": "2026-04-10",
    "currency": "USD",
    "status": "Pending",
    "creditFeeHandling": {
      "payerAbsorbPercentage": 1.0
    },
    "referenceId": "SHIFT-789012",
    "lineItems": [
      {
        "description": "Shift pay - Phoenix Memorial, 7AM-3PM",
        "totalCost": 450.00,
        "quantity": 1,
        "unit": "shift",
        "costPerUnit": 450.00,
        "labels": {
          "shiftId": "SHIFT-789012",
          "facilityId": "FAC-001",
          "payCode": "STANDARD"
        }
      }
    ]
  }'
```

Option B: Direct Entity Token

```
curl -X POST https://stagingapi.wingspan.app/payments/payable \
  -H "Authorization: Bearer {NURSEIO_AZ_API_TOKEN}" \
  -H "Content-Type: application/json" \
```

```
-d '{
  "collaboratorId": "ppe_az_jane_abc123...",
  "dueDate": "2026-04-10",
  "currency": "USD",
  "status": "Pending",
  "creditFeeHandling": {
    "payerAbsorbPercentage": 1.0
  },
  "referenceId": "SHIFT-789012",
  "lineItems": [
    {
      "description": "Shift pay - Phoenix Memorial, 7AM-3PM",
      "totalCost": 450.00,
      "quantity": 1,
      "unit": "shift",
      "costPerUnit": 450.00
    }
  ]
}'
```

Key: The `collaboratorId` is the `payerPayeeEngagementId` from Part 5 — NOT the `payeeId`. This is the engagement-level relationship ID that ties a specific contractor to a specific entity.

Request Body Fields

Field	Type	Req	Description
`collaboratorId`	string	Yes	The payerPayeeEngagementId from Part 5. Identifies the contractor's relationship to the entity.
`currency`	string	Yes	Must be "USD". Required on every payable.
`status`	string	Rec.	Set to "Pending" to create as approved/ready-to-pay. If omitted, defaults to "Draft".
`dueDate`	string	Yes	Payment due date (YYYY-MM-DD). For instant pay, use today.
`creditFeeHandling`	object	Yes	payerAbsorbPercentage: 1.0 = NurseIO absorbs all fees.
`lineItems`	array	Yes	One or more line items describing the payment.
`referenceId`	string	Rec.	Idempotency key — use your shift ID (e.g., SHIFT-789012). Prevents duplicate payables.

Line Item Fields

Field	Type	Req	Description
`description`	string	No	Human-readable description of the line item.
`totalCost`	number	Yes	Dollar amount for this line item.
`quantity`	number	No	Number of units (defaults to 1).
`unit`	string	No	Unit label (e.g., "shift", "hour").
`costPerUnit`	number	No	Cost per unit.
`labels`	object	No	Key-value metadata (shift ID, facility ID, pay code).

`reimbursableExpense`	boolean	No	Whether this is a reimbursable expense.

Response

```
{
  "payableId": "inv_abc123...",
  "collaboratorId": "ppe_az_jane_abc123...",
  "dueDate": "2026-04-10",
  "status": "Pending",
  "lineItems": [
    {
      "description": "Shift pay - Phoenix Memorial, 7AM-3PM",
      "totalCost": 450.00,
      "quantity": 1,
      "unit": "shift",
      "costPerUnit": 450.00
    }
  ],
  "referenceId": "SHIFT-789012"
}
```

Payable Status

You can create the payable directly in `"Pending"` status (approved and ready to pay) by adding `"status": "Pending"` to the request body. If omitted, the payable defaults to `"Draft"` and you'd need a separate PATCH to move it to Open → Pending. For instant pay on clock-out, always create with `status: "Pending"` to skip the extra step.

Step 2: Pay Approved Payables (Bulk Payroll Batch)

Once payables are created, trigger a bulk payroll batch to pay all approved payables for the entity. This call sweeps up every approved payable under the entity and processes them as a single payroll run.

Endpoint:

```
POST https://stagingapi.wingspan.app/payments/pay-approved
```

Option A: Parent Token + X-WINGSPAN-USER (Entity Context)

```
curl -X POST https://stagingapi.wingspan.app/payments/pay-approved \
  -H "Authorization: Bearer {SLINK_HEALTH_PARENT_API_TOKEN}" \
  -H "X-WINGSPAN-USER: {NURSEIO_AZ_USER_ID}" \
  -H "Content-Type: application/json"
```

Option B: Direct Entity Token

```
curl -X POST https://stagingapi.wingspan.app/payments/pay-approved \
  -H "Authorization: Bearer {NURSEIO_AZ_API_TOKEN}" \
  -H "Content-Type: application/json"
```

This endpoint takes no request body — it uses the authenticated entity context to find and batch all approved payables.

Response:

```
{
  "bulkPayrollBatchId": "bpb_abc123...",
  "status": "Pending"
}
```

Funding

NurseIO is using the debit-from-bank-account funding model. When the payroll batch processes, Wingspan debits from the entity's linked bank account and disburses to each contractor. Since payments are instant and unpredictable in timing, this is the right approach.

The Full Flow (Clock-Out to Cash)

16. Nurse Jane clocks out at Phoenix Memorial (NurseIO AZ facility) via NFC
17. Your system determines the entity: NurseIO AZ
18. Your system looks up Jane's payerPayeeEngagementId for NurseIO AZ (saved from Part 5)
19. Creates a payable via POST /payments/payable with collaboratorId = that engagement ID, status = "Pending"
20. Calls POST /payments/pay-approved under the NurseIO AZ context
21. Wingspan batches the approved payable into a payroll run
22. Wingspan debits NurseIO AZ's bank account
23. Jane gets instant payment to her wallet/card — grocery store by 7:19 PM

Timing: When to Call Pay-Approved

- **Immediately after each payable:** Call POST /payments/pay-approved right after creating the payable for true instant pay on every clock-out.
- **On a short interval:** Call every 5-15 minutes per entity to batch up multiple payables. More efficient if multiple nurses clock out in a window.
- **End of shift batch:** Call once at the end of each shift period to process all payables at once.

For NurseIO's instant pay promise, the first option (immediately after each payable) is the most natural fit.

Part 7: Session Token & Embedded Onboarding (Phase 2)

For after Phase 1 is stable. When a new contractor hits step 4 of your onboarding wizard (payment setup), generate a session token and redirect them into Wingspan's hosted onboarding — or embed it directly.

Generate a Session Token

```
GET
https://stagingapi.wingspan.app/users/organization/user/{providerUserId}/session

Authorization: Bearer {SLINK_HEALTH_PARENT_API_TOKEN}
```

Where `{providerUserId}` is the `user.userId` from the payee creation response in Part 4.

- Token expires after 60 minutes
- Use requestingToken in the redirect URL — NOT token (common gotcha)

Redirect URL

```
https://staging-my.wingspan.app/member/onboarding?requestingToken={token}
```

Deep-Link Paths

Destination	Path
Main dashboard	/member
Onboarding flow	/member/onboarding
Payout settings	/member/settings/payment-method
Tax info	/member/settings/tax-info

Embedded SDK (Alternative to Redirect)

```
npm install @wingspanhq/embedded-sdk

import { Wingspan } from "@wingspanhq/embedded-sdk";

const ws = Wingspan.init({
  baseUrl: "https://staging-my.wingspan.app",
  token: "<session-token>",
});

const onboarding = ws.createOnboarding({
  modules: [
    { type: "create_account", accountType: "payee" },
    { type: "certify_tax_payer_info" },
    { type: "payout_method" },
  ],
  onComplete: (data) => console.log("Done!", data),
  onError: (error) => console.error(error),
});
```

```
onboarding.openModal();
```

30-Day Implementation Timeline

Based on the kickoff call, here's the target path from today to live. Josh mentioned wanting to be live within 30 days — this plan is designed to hit that.

Week 1: Sandbox Setup & Configuration (Days 1-7)

- Wingspan provisions staging org structure (Slink Health → Payee Bucket + Entity accounts)
- NurseIO generates staging API token and confirms access
- Wingspan configures inheritance, branding, terminology, support
- NurseIO reviews appearance config in staging UI
- NurseIO wires up API connectivity from dev environment to staging
- **Milestone:** Can successfully call POST /payments/payee and see a contractor in the staging UI

Week 2: Core API Integration (Days 8-14)

- Implement payee creation (Part 4) — triggered on contractor activation
- Implement entity association (Part 5) — triggered on first shift assignment
- Implement payable creation via individual endpoint with collaboratorId (Part 6) — triggered on NFC clock-out
- Implement pay-approved trigger (Part 6 Step 2) — triggered after payable creation
- Test end-to-end: payee → entity association → payable → payment confirmation
- **Milestone:** 5-10 test contractors onboarded, payables processing across 2+ entities

Week 3: Testing & Validation (Days 15-21)

- Run 50+ contractors through the full flow in staging
- Test cross-entity scenarios (contractor works under AZ and KS)
- Test error handling (duplicate payee, invalid external ID, failed batch items)
- Test session token generation and embedded onboarding (Phase 2 prep)
- Load-test payable creation at expected volume (~1,000 contractors, ~15 new/day)
- **Milestone:** Staging fully validated, error handling confirmed, volume tested

Week 4: Migration & Go-Live (Days 22-30)

- Wingspan provisions production org structure (mirrors staging)
- NurseIO swaps staging URLs → production URLs in their codebase
- NurseIO generates production API token
- Bulk-migrate existing contractors: create payees, establish entity relationships
- Validate first 10-20 production payables with real contractors
- Monitor first few days of live traffic for edge cases
- **Milestone:** Live in production, initial contractors migrated, instant pay flowing

Key Dependencies

Item	Owner	By When
Staging org provisioning	Wingspan	End of Week 1

API token generation and test call	NurseIO (Josh/Lemuel/Arun)	End of Week 1
Core API integration (Parts 4-6)	NurseIO Engineering	End of Week 2
Entity list confirmed (~5 with EINs)	NurseIO (Josh)	End of Week 1
Bank accounts linked per entity	NurseIO + Wingspan	End of Week 2
Staging validation sign-off	Both teams	End of Week 3
Production provisioning	Wingspan	Start of Week 4
Bulk migration plan	Both teams	Start of Week 4

Common Errors

Code	Cause	Fix
401	Invalid or expired token	Regenerate in Data & Integrations > Developer
403	Token can't access this child	Verify X-WINGSPAN-USER matches the correct child User ID
404	Resource not in org hierarchy	An external ID from one entity won't resolve under another — check context
409	Duplicate payee (same email, same entity)	Safe to ignore — relationship already exists. GET to retrieve.
422	Validation error	Check response body for the specific field
429	Rate limit exceeded	Implement exponential backoff

Staging Checklist

- ☐ Generate API token from root parent (Slink Health) account
- ☐ Confirm Payee Bucket child account is created — save its User ID
- ☐ Confirm at least one entity child account (e.g., NurseIO AZ) is created — save its User ID
- ☐ Create a test payee in the Payee Bucket via POST /payments/payee with a payeeExternalId — save user.userId
- ☐ Establish entity relationship: POST /payments/payee from entity context with same email and same payeeExternalId — save the payerPayeeEngagementId
- ☐ Create a test payable via POST /payments/payable using the payerPayeeEngagementId as collaboratorId
- ☐ Trigger payment via POST /payments/pay-approved from the entity context — verify payroll batch processes
- ☐ Generate a session token and confirm redirect to onboarding works
- ☐ Set up appearance config (branding, terminology) and verify contractor-facing UI
- ☐ Share test IDs with Jason for validation
- ☐ Test with 5-10 contractors end-to-end across at least 2 entities before migration

ID Reference (Save These)

Keep a mapping of these IDs — your backend will need them:

ID	Source	Used For
Slink Health User ID	Root parent account	API token, appearance config, org account creation
Slink Health Org ID	Root parent account	organizationId in org account creation (Part 1)
Payee Bucket User ID	Child account creation	X-WINGSPAN-USER when creating payees (Part 4)
NurseIO AZ User ID	Child account creation	X-WINGSPAN-USER when creating payables for AZ
NurseIO KS User ID	Child account creation	X-WINGSPAN-USER when creating payables for KS
... (per entity)	Child account creation	Same pattern per entity
payeeExternalId	Your system (e.g., NIO-123456)	Set during payee creation (Parts 4 & 5) for lookups — YOUR ID, not Wingspan's
payerPayeeEngagementId	Part 5 response	Used as collaboratorId in Part 6 payable creation. Save per entity per contractor.
user.userId	Part 4 response	Session token generation (Part 7)

Next Steps

24. **Immediate:** Review this guide, generate your staging API token, create a test payee + entity relationship + payable.
25. **Technical Call (April 10):** Jason walks you through sandbox setup, creates entity child accounts, and answers API questions. Bring Lemuel and Arun. We'll confirm entity count (~5?).
26. **Phase 1 — New Contractors:** Wire up payee creation (Payee Bucket) on activation. Entity association on first shift assignment (save the payerPayeeEngagementId!). Payable creation + pay-approved trigger on NFC clock-out.
27. **Validate:** Run 10-20 contractors across 2+ entities through the full flow.
28. **Production:** When staging is validated, we provision prod with the same org structure — swap URLs.